AI Applications Lecture 15

Iterative Latent Generation: Stable Diffusion 1.5 U-Net and Qwen-Image-2.1 DiT

SUZUKI, Atsushi[a]

[a] I thank my collaborator, Jing WANG, for cooperating in creating the teaching materials.

Outline

1. Introduction

2. Mathematical notation

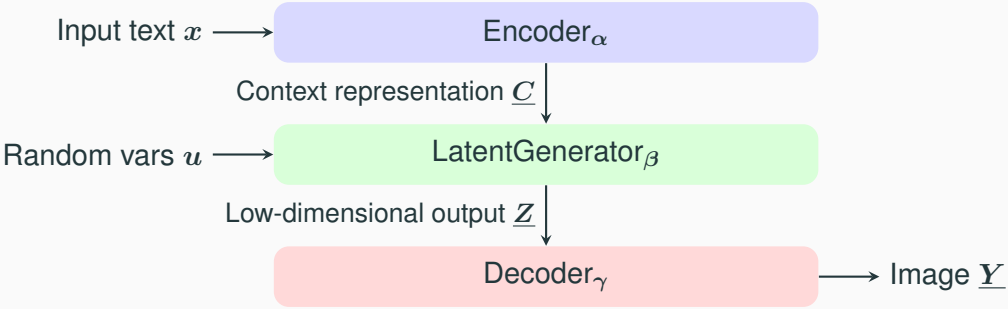
3. Text encoder of Stable Diffusion 1.5

4. Can’t we use vector operations to modify the prompt information? - Prompt weighting.

5. Text encoder of Qwen-Image-2.1

6. Summary

Review the latent generation framework



Example iterative generation: Let $w_0 = \epsilon$. For $k = 0, 1, ..., K-1$:

- **Reverse diffusion (DDIM)**:
  $w_{k+1} \leftarrow \frac{b(t_{k+1})}{b(t_k)} w_k + \left[a(t_{k+1}) - \frac{a(t_k) b(t_{k+1})}{b(t_k)} \right] \widehat{\epsilon}_\theta(w_k, c, t_k)$.
- **Flow matching (linear path + Euler)**: $w_{k+1} \leftarrow w_k + (t_{k+1} - t_k) \widehat{v}_\theta(w_k, c, t_k)$.

Learning outcomes

At the end of this lecture, you will be able to:

Learning outcomes

At the end of this lecture, you will be able to:

1. **Explain** the architecture of the text encoder in SD 1.5, and why it has a prompt length constraint.

Learning outcomes

At the end of this lecture, you will be able to:

1. **Explain** the architecture of the text encoder in SD 1.5, and why it has a prompt length constraint.
2. **Modify** the information from a user prompt to the iterative generator using vector operations.

Learning outcomes

At the end of this lecture, you will be able to:

1. **Explain** the architecture of the text encoder in SD 1.5, and why it has a prompt length constraint.
2. **Modify** the information from a user prompt to the iterative generator using vector operations.
3. **Design** a token encoder that does not limit the prompt length, taking Qwen-Image-2.1's text encoder as an example.

Outline

1. Introduction

2. Mathematical notation

3. Text encoder of Stable Diffusion 1.5

4. Can’t we use vector operations to modify the prompt information? - Prompt weighting.

5. Text encoder of Qwen-Image-2.1

6. Summary

Outline

1. Introduction

2. Mathematical notation

3. Text encoder of Stable Diffusion 1.5

4. Can’t we use vector operations to modify the prompt information? - Prompt weighting.

5. Text encoder of Qwen-Image-2.1

6. Summary

Preliminaries: mathematical notations

Set:

- Sets: $\mathcal{A}$
- Membership: $x \in \mathcal{A}$
- Empty set: $\{\}$
- Roster notation: $\{a, b, c\}$
- Set-builder: $\{x \in \mathcal{A} \mid P(x)\}$
- Cardinality: $|\mathcal{A}|$
- Real numbers: $\mathbb{R}, \mathbb{R}_{>0}, \mathbb{R}_{\geq 0}$
- Integers: $\mathbb{Z}, \mathbb{Z}_{>0}, \mathbb{Z}_{\geq 0}$
- Integer range: $[1, k]_{\mathbb{Z}} \coloneqq \{1, \ldots, k\}$

Function:

- $f : \mathcal{X} \to \mathcal{Y}$: f maps from $\mathcal{X}$ to $\mathcal{Y}$.
- $y = f(x)$: The output of f for input x.

Definition:

- $(\mathrm{LHS}) \coloneqq (\mathrm{RHS})$: Left side is defined by the right side.

Preliminaries: mathematical notations

Sequence: Denoted by $\boldsymbol{a} = (a_1, a_2, \dots)$.

- A function $\boldsymbol{a} : [1, n]_{\mathbb{Z}} \to \mathcal{A}$.
- Length is denoted by $|\boldsymbol{a}|$.

Vector: Denoted by $\boldsymbol{v}$.

- A column of numbers, $\boldsymbol{v} \in \mathbb{R}^n$.
- i-th element is v_i.

Matrix: Denoted by $\boldsymbol{A}$.

- $m \times n$ matrix: $\boldsymbol{A} \in \mathbb{R}^{m,n}$.
- (i, j)-th element is $a_{i,j}$.
- Transpose: $\boldsymbol{A}^\top$.

Tensor: Denoted by $\underline{\boldsymbol{A}}$.

- Simply a multi-dimensional array.
- Vector $\to$ 1st-order, Matrix $\to$ 2nd-order.

Outline

1. Introduction

2. Mathematical notation

3. Text encoder of Stable Diffusion 1.5

4. Can’t we use vector operations to modify the prompt information? - Prompt weighting.

5. Text encoder of Qwen-Image-2.1

6. Summary

Outline

1. Introduction

2. Mathematical notation

3. Text encoder of Stable Diffusion 1.5

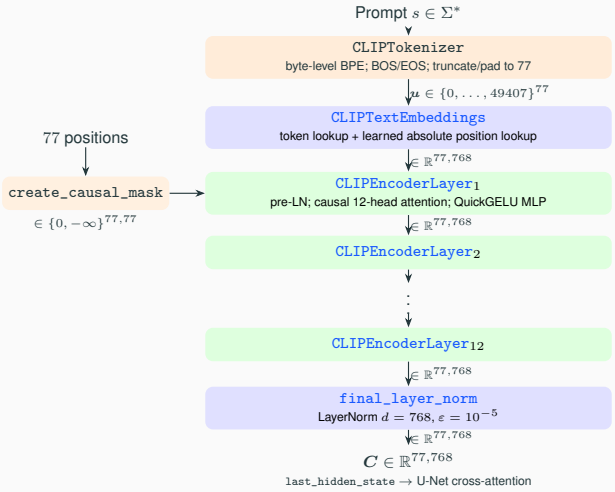
4. Can’t we use vector operations to modify the prompt information? - Prompt weighting.

5. Text encoder of Qwen-Image-2.1

6. Summary

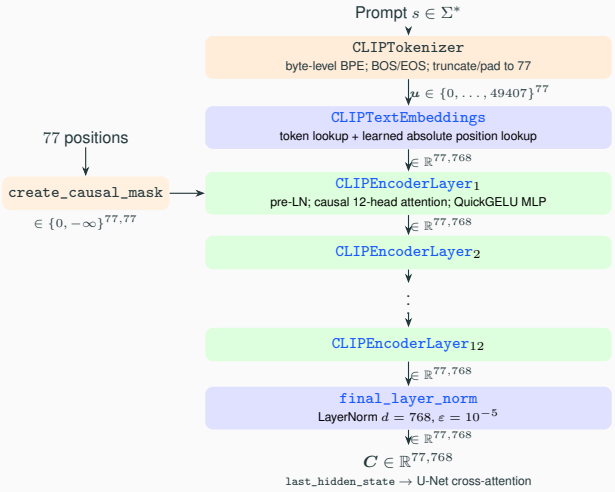
Question: How the text encoder convert users' prompt for the iterative generator? Why is it clipped to 77 tokens?

Stable Diffusion 1.5 text encoder: CLIP ViT-L/14 — Whole structure



The structure is simple:

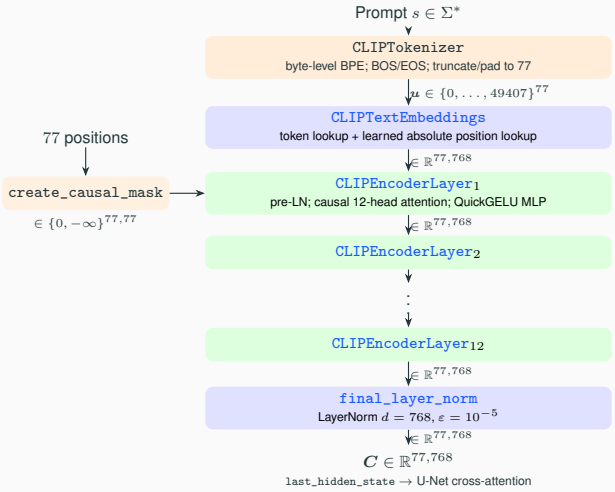
Stable Diffusion 1.5 text encoder: CLIP ViT-L/14 — Whole structure



The structure is simple:

- Truncate/padding to 77 tokens

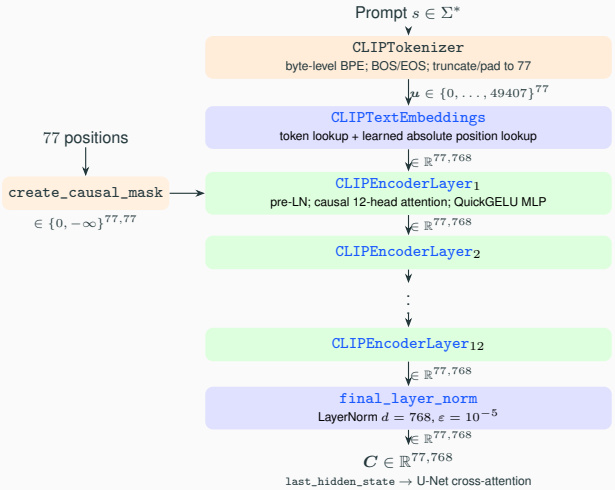
Stable Diffusion 1.5 text encoder: CLIP ViT-L/14 — Whole structure



The structure is simple:

- Truncate/padding to 77 tokens
- Embedding

Stable Diffusion 1.5 text encoder: CLIP ViT-L/14 — Whole structure

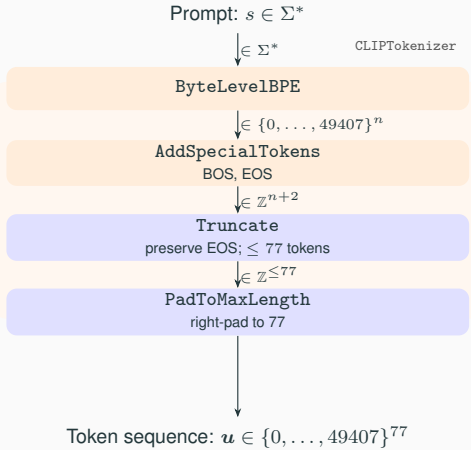


Source: `modeling_clip.py`; `pipeline_stable_diffusion.py`; SD1.5 text config; `tokenization_clip.py`.

The structure is simple:

- Truncate/padding to 77 tokens
- Embedding
- Attention layers

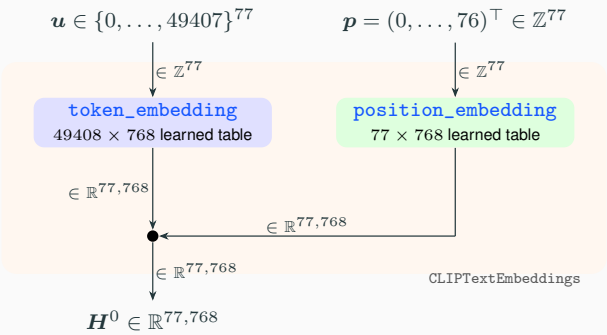
CLIPTokenizer



Source: `pipeline_stable_diffusion.py`; `tokenization_clip.py`; SD1.5 text config.

Apply truncating and padding, to get 77 tokens.

CLIPTextEmbeddings



Learnable parameters are $\boldsymbol{W}_{\text{token}} \in \mathbb{R}^{49408,768}$ and $\boldsymbol{W}_{\text{position}} \in \mathbb{R}^{77,768}$. For each token position $i = 0, \ldots, 76$, the output $\boldsymbol{h}_i$ is given by

$$\boldsymbol{h}_i = \boldsymbol{w}_{\text{token},u_i} + \boldsymbol{w}_{\text{Position},i}.$$

Source: `modeling_clip.py`; SD1.5 text config.

CLIPTextEmbeddings — exercise

Exercise

Let the embedding dimension be $d = 2$, the vocabulary size be $V = 3$ (token IDs $0, 1, 2$), and the maximum sequence length be 4. The learned embedding tables are

$$\boldsymbol{W}_{\text{token}} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 2 & 1 \end{pmatrix} \in \mathbb{R}^{3,2}, \qquad \boldsymbol{W}_{\text{position}} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 2 \\ -1 & 1 \end{pmatrix} \in \mathbb{R}^{4,2}.$$

At position $i = 1, \ldots, 4$, add the token embedding in row $u_i + 1$ to the position embedding in row i.

1. *For $\boldsymbol{u} = (0, 2, 1, 1)^\top$, compute $\boldsymbol{h}_i \in \mathbb{R}^2$ for all i and $\boldsymbol{H}^0 \in \mathbb{R}^{4,2}$.*
2. *For $\boldsymbol{u} = (0, 2, 1, 1, 2)^\top$, determine whether the same operation defines $\boldsymbol{H}^0 \in \mathbb{R}^{5,2}$, and explain why.*

CLIPTextEmbeddings — answer

Answer:

CLIPTextEmbeddings — answer

Answer: For $\boldsymbol{u} = (0, 2, 1, 1)^\top$, take the following four rows from each table.

CLIPTextEmbeddings — answer

Answer: For $\boldsymbol{u} = (0, 2, 1, 1)^\top$, take the following four rows from each table.

Token lookup $(0, 2, 1, 1)$

$$\begin{pmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 1 \\ 0 & 1 \end{pmatrix} \in \mathbb{R}^{4,2}$$

CLIPTextEmbeddings — answer

Answer: For $\boldsymbol{u} = (0, 2, 1, 1)^\top$, take the following four rows from each table.

Token lookup $(0, 2, 1, 1)$

$$\begin{pmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 1 \\ 0 & 1 \end{pmatrix} \in \mathbb{R}^{4,2}$$

Position lookup $(0, 1, 2, 3)$

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 2 \\ -1 & 1 \end{pmatrix} \in \mathbb{R}^{4,2}$$

CLIPTextEmbeddings — answer

Answer: For $\boldsymbol{u} = (0, 2, 1, 1)^\top$, take the following four rows from each table.

Token lookup $(0, 2, 1, 1)$

$$\begin{pmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 1 \\ 0 & 1 \end{pmatrix} \in \mathbb{R}^{4,2}$$

Position lookup $(0, 1, 2, 3)$

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 2 \\ -1 & 1 \end{pmatrix} \in \mathbb{R}^{4,2}$$

Rowwise sum

$$\boldsymbol{H}^0 = \begin{pmatrix} 1 & 1 \\ 3 & 1 \\ 0 & 3 \\ -1 & 2 \end{pmatrix} \in \mathbb{R}^{4,2}$$

CLIPTextEmbeddings — answer

Answer: For $\boldsymbol{u} = (0, 2, 1, 1)^\top$, take the following four rows from each table.

Token lookup $(0, 2, 1, 1)$

$$\begin{pmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 1 \\ 0 & 1 \end{pmatrix} \in \mathbb{R}^{4,2}$$

Position lookup $(0, 1, 2, 3)$

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 2 \\ -1 & 1 \end{pmatrix} \in \mathbb{R}^{4,2}$$

Rowwise sum

$$\boldsymbol{H}^0 = \begin{pmatrix} 1 & 1 \\ 3 & 1 \\ 0 & 3 \\ -1 & 2 \end{pmatrix} \in \mathbb{R}^{4,2}$$

$$\boldsymbol{h}_1 = (1,0)^\top + (0,1)^\top = (1,1)^\top, \quad \boldsymbol{h}_2 = (2,1)^\top + (1,0)^\top = (3,1)^\top,$$
$$\boldsymbol{h}_3 = (0,1)^\top + (0,2)^\top = (0,3)^\top, \quad \boldsymbol{h}_4 = (0,1)^\top + (-1,1)^\top = (-1,2)^\top.$$

CLIPTextEmbeddings — answer

Answer: For $\boldsymbol{u} = (0, 2, 1, 1)^\top$, take the following four rows from each table.

Token lookup $(0, 2, 1, 1)$

$$\begin{pmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 1 \\ 0 & 1 \end{pmatrix} \in \mathbb{R}^{4,2}$$

Position lookup $(0, 1, 2, 3)$

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 2 \\ -1 & 1 \end{pmatrix} \in \mathbb{R}^{4,2}$$

Rowwise sum

$$\boldsymbol{H}^0 = \begin{pmatrix} 1 & 1 \\ 3 & 1 \\ 0 & 3 \\ -1 & 2 \end{pmatrix} \in \mathbb{R}^{4,2}$$

$$\boldsymbol{h}_1 = (1,0)^\top + (0,1)^\top = (1,1)^\top, \quad \boldsymbol{h}_2 = (2,1)^\top + (1,0)^\top = (3,1)^\top,$$
$$\boldsymbol{h}_3 = (0,1)^\top + (0,2)^\top = (0,3)^\top, \quad \boldsymbol{h}_4 = (0,1)^\top + (-1,1)^\top = (-1,2)^\top.$$

CLIPTextEmbeddings — answer

For $\boldsymbol{u} = (0, 2, 1, 1, 2)^\top$, the fifth token embedding $(2, 1)^\top$ exists, but the fifth position embedding does not: $\boldsymbol{W}_{\text{position}} \in \mathbb{R}^{4,2}$ has only four rows. Thus $\boldsymbol{h}_5$ and an output in $\mathbb{R}^{5,2}$ are **undefined**.

The learnable position embedding is the cause of 77-token limitation

The text embedding of CLIP in SD 1.5 involves $\boldsymbol{W}_{\text{token}} \in \mathbb{R}^{49408,768}$ and $\boldsymbol{W}_{\text{position}} \in \mathbb{R}^{77,768}$.

Specifically,

The learnable position embedding is the cause of 77-token limitation

The text embedding of CLIP in SD 1.5 involves $\boldsymbol{W}_{\text{token}} \in \mathbb{R}^{49408,768}$ and $\boldsymbol{W}_{\text{position}} \in \mathbb{R}^{77,768}$.

Specifically, we need to look up the i-th vector in $\boldsymbol{W}_{\text{position}}$. However,

The learnable position embedding is the cause of 77-token limitation

The text embedding of CLIP in SD 1.5 involves $\boldsymbol{W}_{\text{token}} \in \mathbb{R}^{49408,768}$ and $\boldsymbol{W}_{\text{position}} \in \mathbb{R}^{77,768}$.

Specifically, we need to look up the i-th vector in $\boldsymbol{W}_{\text{position}}$. However, it only learn 77 embedding vectors in the training phase.

This is why

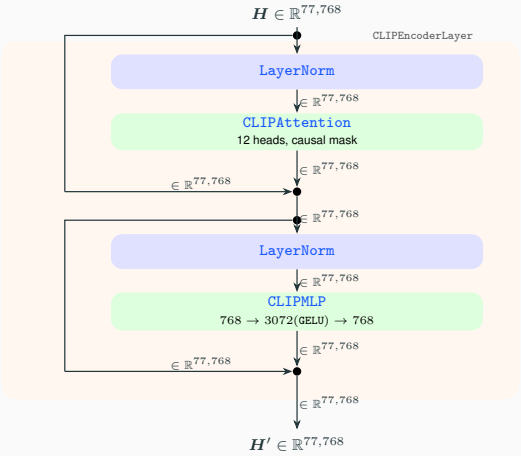
The learnable position embedding is the cause of 77-token limitation

The text embedding of CLIP in SD 1.5 involves $\boldsymbol{W}_{\text{token}} \in \mathbb{R}^{49408,768}$ and $\boldsymbol{W}_{\text{position}} \in \mathbb{R}^{77,768}$.

Specifically, we need to look up the i-th vector in $\boldsymbol{W}_{\text{position}}$. However, it only learn 77 embedding vectors in the training phase.

This is why we need to clip the input prompt to 77 tokens.

CLIPEncoderLayer (each of 12)



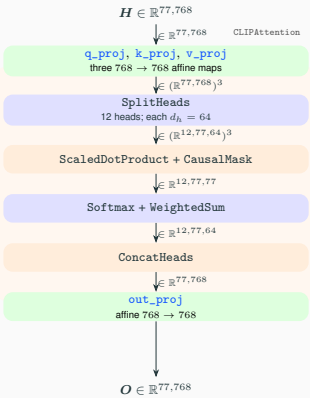
Input from: CLIPTextEmbeddings or preceding CLIPEncoderLayer.

Output to: next CLIPEncoderLayer or final LayerNorm.

Both residual additions receive the corresponding unnormalized branch input. The causal mask is supplied to `self_attn`. $\varepsilon = 10^{-5}$. Twelve different parameter sets.

Source: `CLIPEncoderLayer`, L2701–2752; `CLIPAttention`, L2590–2676; `CLIPMLP`, L2678–2699; SD1.5 text config.

CLIPAttention



Source: `modeling_clip.py`; SD1.5 text config; Vaswani et al. (2017), Fig. 2.

Input from: CLIPEncoderLayer/`layer_norm1`.

Output to: first residual addition.

In each head, query i attends only to positions $j \leq i$. Attention weights: $12 \times 77 \times 77$. Linear projections include learned biases. SD1.5 sets attention dropout to 0.

CLIP causal attention mask

For sequence length $T \leq 77$, define the additive mask $\boldsymbol{A} \in (\mathbb{R} \cup \{-\infty\})^{T,T}$ by

CLIP causal attention mask

For sequence length $T \leq 77$, define the additive mask $\boldsymbol{A} \in (\mathbb{R} \cup \{-\infty\})^{T,T}$ by

$$A_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

CLIP causal attention mask

For sequence length $T \leq 77$, define the additive mask $\boldsymbol{A} \in (\mathbb{R} \cup \{-\infty\})^{T,T}$ by

$$A_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

At query token i, key token j is accessible precisely when $j \leq i$. The `CLIPTextModel` constructs this mask before its 12 encoder layers.

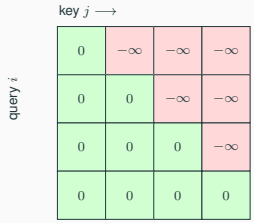
CLIP causal attention mask

For sequence length $T \leq 77$, define the additive mask $\boldsymbol{A} \in (\mathbb{R} \cup \{-\infty\})^{T,T}$ by

$$A_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

At query token i, key token j is accessible precisely when $j \leq i$. The `CLIPTextModel` constructs this mask before its 12 encoder layers. This is a causal text mask, not image block-causal attention.

Source: `modeling_clip.py`; Vaswani et al. (2017), Fig. 2.



How do we train the CLIP encoder?

How do we train the CLIP encoder?

We train the text encoder NN and image encoder NN on the **contrastive learning** scheme.

How do we train the CLIP encoder?

We train the text encoder NN and image encoder NN on the **contrastive learning** scheme.

In fact, CLIP refers to **Contrastive Language – Image Pre-training** [2].

Since the actual training source code used for the CLIP ViT-L/14 in Stable Diffusion 1.5 does not seem to be publicly available, we will explain based on `open_clip` (`mlfoundations/open_clip`) OpenCLIP repository: `https://github.com/mlfoundations/open_clip`.

CLIP Architecture and Objective Function i

Consider text sequences and image sequences of batch size N, $(\boldsymbol{t}_i,\ \underline{\boldsymbol{I}}_i)_{i=1}^N$.

- **Text Encoder** $\mathsf{TextEncoder}_{\boldsymbol{\theta}_{\mathrm{TE}}}:\ \{1,2,...,V\}^L \to \mathbb{R}^d$.
- **Image Encoder** $\mathsf{ImageEncoder}_{\boldsymbol{\theta}_{\mathrm{IE}}}:\ \mathcal{X}_{\mathrm{img}} \to \mathbb{R}^d$ (ViT/ResNet etc. [1]).
- **Temperature** $\tau > 0$ (often made learnable in implementations).

For each sample pair $(\boldsymbol{t}_i,\ \underline{\boldsymbol{I}}_i)$,

$$\boldsymbol{u}_i = \frac{\mathsf{TextEncoder}_{\boldsymbol{\theta}_{\mathrm{TE}}}(\boldsymbol{t}_i)}{\|\mathsf{TextEncoder}_{\boldsymbol{\theta}_{\mathrm{TE}}}(\boldsymbol{t}_i)\|_2}, \qquad \boldsymbol{v}_i = \frac{\mathsf{ImageEncoder}_{\boldsymbol{\theta}_{\mathrm{IE}}}(\underline{\boldsymbol{I}}_i)}{\|\mathsf{ImageEncoder}_{\boldsymbol{\theta}_{\mathrm{IE}}}(\underline{\boldsymbol{I}}_i)\|_2} \in \mathbb{R}^d. \tag{1}$$

CLIP Architecture and Objective Function ii

The similarity matrix $\boldsymbol{S} \in \mathbb{R}^{N \times N}$ is

$$S_{ij} = \frac{\langle \boldsymbol{u}_i, \boldsymbol{v}_j \rangle}{\tau}. \tag{2}$$

The **cross-entropy losses** for text→image and image→text are

$$\mathcal{L}_{\text{Text}\to\text{Image}} = \frac{1}{N} \sum_{i=1}^{N} \Big(- \log \frac{\exp(S_{ii})}{\sum_{j=1}^{N} \exp(S_{ij})} \Big), \tag{3}$$

$$\mathcal{L}_{\text{Image}\to\text{Text}} = \frac{1}{N} \sum_{i=1}^{N} \Big(- \log \frac{\exp(S_{ii})}{\sum_{j=1}^{N} \exp(S_{ji})} \Big), \tag{4}$$

CLIP Architecture and Objective Function iii

and the **symmetric loss**

$$\mathcal{L}_{\text{CLIP}}(\boldsymbol{\theta}_{\text{TE}}, \boldsymbol{\theta}_{\text{IE}}, \tau) = \frac{1}{2}\left(\mathcal{L}_{\text{Text}\to\text{Image}} + \mathcal{L}_{\text{Image}\to\text{Text}}\right) \tag{5}$$

is **minimized**.

Similarity matrix

Similarity matrix $S = \left[\langle \boldsymbol{u}_i, \boldsymbol{v}_j \rangle / \tau\right]_{i,j=1}^{N}$ (Ideal state)

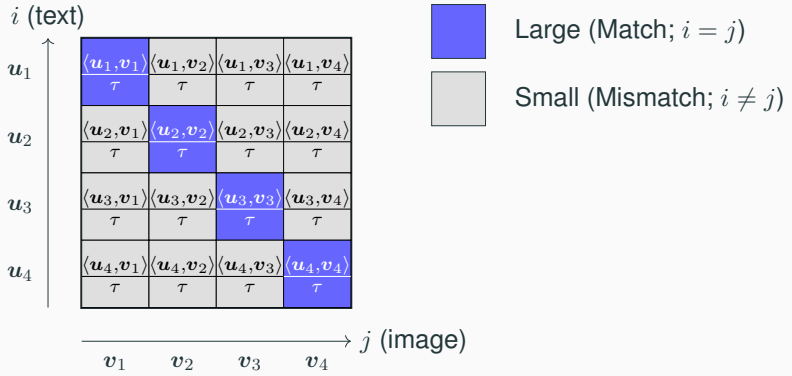


Figure 1: Visualization of the ideal similarity matrix.

Intuition of CLIP

Remark

Equation (5) *applies pressure to make the inner product (cosine similarity) of the corresponding pair* (i, i) ***larger than others****. The temperature* τ *controls the sharpness of the softmax, and as* $\tau \downarrow 0$*, it approaches harder discrimination [2]. The OpenCLIP implementation conforms to this formulation*[1] *.*

[1] The OpenCLIP loss implementation is included in the training code at `https://github.com/mlfoundations/open_clip`.

Text encoder in SD 1.5 - Answer

Question: How the text encoder convert users' prompt for the iterative generator? Why is it clipped to 77 tokens?

Answer:

- The text encoder of SD 1.5 uses CLIP encoder. The architecture consists of Transformer layers.
- It learns parameters on contrastive learning.
- Since it uses learnable position embedding parameters, it cannot accept a prompt longer than 77 tokens.

Outline

1. Introduction

2. Mathematical notation

3. Text encoder of Stable Diffusion 1.5

4. Can’t we use vector operations to modify the prompt information? - Prompt weighting.

5. Text encoder of Qwen-Image-2.1

6. Summary

Outline

1. Introduction

2. Mathematical notation

3. Text encoder of Stable Diffusion 1.5

4. Can’t we use vector operations to modify the prompt information? - Prompt weighting.

5. Text encoder of Qwen-Image-2.1

6. Summary

Question: We convert text prompt information to a series of vectors by the text encoder. Can't we

Question: We convert text prompt information to a series of vectors by the text encoder. Can't we modify the information using vector operations?

Basic Idea

The text encoder outputs a tensor, which is passed to the iterative generator.

Prompt weighting

Basic Idea

The text encoder outputs a tensor, which is passed to the iterative generator.

Prompt weighting is a technique that processes this tensor between the text encoder and the iterative, reflecting user intentions that cannot be fully expressed in the natural language input to the text encoder.

Compel syntax examples:

When using Diffusers for image generation using Python, you can use the `Compel` library to control the context representation vectors.

Compel syntax examples:

When using Diffusers for image generation using Python, you can use the `Compel` library to control the context representation vectors.

Original prompt: `A red dog is running on the moon.`

Compel syntax examples:

When using Diffusers for image generation using Python, you can use the `Compel` library to control the context representation vectors.

Original prompt: `A red dog is running on the moon.`

When emphasizing "red": `A red+ dog is running on the moon.`

Compel syntax for prompt weighting

Compel assigns a scalar weight r to a word or a parenthesized text span.

Compel syntax	Weight r
`span+`, `(two words)+`	1.1
`span++`, `(two words)++`	1.1^2
`span-`, `(two words)-`	0.9
`span--`, `(two words)--`	0.9^2
`(two words)1.4`	1.4
`(two words)0.6`	0.6

`(span)` groups words; the suffix sets r. Nested weights multiply: `(blue eyes+)++` gives 1.1^3 for `eyes`. Without a suffix, $r = 1$.

Example: `(cinematic lighting)1.4, soft focus`: $r_i = 1.4$ for `cinematic lighting` tokens and $r_i = 1$ otherwise.

Specific operations on context vectors

For Stable Diffusion 1.5, let $\boldsymbol{C} = [\boldsymbol{c}_1^\top; \ldots; \boldsymbol{c}_{77}^\top] \in \mathbb{R}^{77,768}$ be the CLIP output, and let $\boldsymbol{c}_i^\varnothing \in \mathbb{R}^{768}$ be the corresponding vector from the empty prompt.

For the per-token weight $r_i > 1$, the Compel converts the corresponding vector as follows:

$$\widehat{\boldsymbol{c}}_i = \boldsymbol{c}_i^\varnothing + r_i(\boldsymbol{c}_i - \boldsymbol{c}_i^\varnothing) \in \mathbb{R}^{768}, \quad 1 \leq i \leq 77.$$

Thus r_i scales the **difference from the empty-prompt vector**, not $\boldsymbol{c}_i$ itself.

Source: Compel, `embeddings_provider.py`, lines 109–118, 146–213, 435–492; README, “1.0.0 - new downweighting algorithm”.

Down-weighting is more complicated

Down-weighting ($0 < r_i < 1$). Compel also computes $\boldsymbol{c}_i^{\text{mask}} \in \mathbb{R}^{768}$ by re-running CLIP with the attention-mask masking the corresponding tokens.

Then, the converted representation vector $\widehat{\boldsymbol{c}}_i$ is given by

$$\widehat{\boldsymbol{c}}_i = \frac{\boldsymbol{c}_i + \alpha(r_i)\boldsymbol{c}_i^{\text{mask}}}{1 + \alpha(r_i)}, \qquad \alpha(r_i) = \tan\left(\frac{\pi(1 - r_i)}{2}\right).$$

Mixed prompt weighting — exercise

Exercise

Consider a six-token example with two-dimensional context vectors. The per-token weights are

$$\boldsymbol{r} = (1, 2, 1, \tfrac{1}{2}, 1, \tfrac{1}{2})^\top \in \mathbb{R}^6.$$

Thus token 2 is up-weighted; tokens 4 and 6 are down-weighted. Suppose the following CLIP outputs have been obtained (rows are token vectors):

$$\boldsymbol{C} = \begin{pmatrix} 1 & 3 \\ 2 & 1 \\ 3 & 1 \\ 5 & 3 \\ 1 & 5 \\ 3 & 5 \end{pmatrix}, \qquad \boldsymbol{C}^{\varnothing} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix} \in \mathbb{R}^{6,2}. \quad \boldsymbol{C}^{(-4)} = \begin{pmatrix} 1 & 3 \\ 2 & 1 \\ 3 & 1 \\ 1 & 1 \\ 3 & 3 \\ 5 & 1 \end{pmatrix}, \quad \boldsymbol{C}^{(-6)} = \begin{pmatrix} 1 & 3 \\ 2 & 1 \\ 3 & 1 \\ 5 & 3 \\ 1 & 5 \\ 1 & 1 \end{pmatrix} \in \mathbb{R}^{6,2}.$$

Here $\boldsymbol{C}^{(-4)}$ and $\boldsymbol{C}^{(-6)}$ are CLIP outputs with token 4 or token 6, respectively, disabled by an attention mask (not simultaneously). Compute the final matrix $\widehat{\boldsymbol{C}} \in \mathbb{R}^{6,2}$ using Compel's default `MASK` down-weighting: first apply each $\boldsymbol{a}_i$ relative to $\boldsymbol{C}^{\varnothing}$, then mix the

Mixed prompt weighting — answer

Answer:

Mixed prompt weighting — answer

Answer: For $s \in \{0, -4, -6\}$, set $\boldsymbol{C}^{(0)} = \boldsymbol{C}$ and calculate, for each $i = 1, \ldots, 6$,

Mixed prompt weighting — answer

Answer: For $s \in \{0, -4, -6\}$, set $\boldsymbol{C}^{(0)} = \boldsymbol{C}$ and calculate, for each $i = 1, \ldots, 6$,

$$\boldsymbol{d}_i^{(s)} = \boldsymbol{c}_i^{\varnothing} + r_i(\boldsymbol{c}_i^{(s)} - \boldsymbol{c}_i^{\varnothing}) \in \mathbb{R}^2.$$

Mixed prompt weighting — answer

Answer: For $s \in \{0, -4, -6\}$, set $\boldsymbol{C}^{(0)} = \boldsymbol{C}$ and calculate, for each $i = 1, \ldots, 6$,

$$\boldsymbol{d}_i^{(s)} = \boldsymbol{c}_i^{\varnothing} + r_i(\boldsymbol{c}_i^{(s)} - \boldsymbol{c}_i^{\varnothing}) \in \mathbb{R}^2.$$

For example, $\boldsymbol{d}_2^{(0)} = (1,1)^\top + 2((2,1)^\top - (1,1)^\top) = (3,1)^\top$.

Mixed prompt weighting — answer

Answer: For $s \in \{0, -4, -6\}$, set $\boldsymbol{C}^{(0)} = \boldsymbol{C}$ and calculate, for each $i = 1, \ldots, 6$,

$$\boldsymbol{d}_i^{(s)} = \boldsymbol{c}_i^{\varnothing} + r_i(\boldsymbol{c}_i^{(s)} - \boldsymbol{c}_i^{\varnothing}) \in \mathbb{R}^2.$$

For example, $\boldsymbol{d}_2^{(0)} = (1,1)^\top + 2((2,1)^\top - (1,1)^\top) = (3,1)^\top$.

$$\boldsymbol{D}^{(0)} = \begin{pmatrix} 1 & 3 \\ 3 & 1 \\ 3 & 1 \\ 3 & 2 \\ 1 & 5 \\ 2 & 3 \end{pmatrix}, \quad \boldsymbol{D}^{(-4)} = \begin{pmatrix} 1 & 3 \\ 3 & 1 \\ 3 & 1 \\ 1 & 1 \\ 3 & 3 \\ 3 & 1 \end{pmatrix}. \qquad \boldsymbol{D}^{(-6)} = \begin{pmatrix} 1 & 3 \\ 3 & 1 \\ 3 & 1 \\ 3 & 2 \\ 1 & 5 \\ 1 & 1 \end{pmatrix}.$$

Mixed prompt weighting — answer

Answer: For $s \in \{0, -4, -6\}$, set $\boldsymbol{C}^{(0)} = \boldsymbol{C}$ and calculate, for each $i = 1, \ldots, 6$,

$$\boldsymbol{d}_i^{(s)} = \boldsymbol{c}_i^{\varnothing} + r_i(\boldsymbol{c}_i^{(s)} - \boldsymbol{c}_i^{\varnothing}) \in \mathbb{R}^2.$$

For example, $\boldsymbol{d}_2^{(0)} = (1,1)^\top + 2((2,1)^\top - (1,1)^\top) = (3,1)^\top$.

$$\boldsymbol{D}^{(0)} = \begin{pmatrix} 1 & 3 \\ 3 & 1 \\ 3 & 1 \\ 3 & 2 \\ 1 & 5 \\ 2 & 3 \end{pmatrix}, \quad \boldsymbol{D}^{(-4)} = \begin{pmatrix} 1 & 3 \\ 3 & 1 \\ 3 & 1 \\ 1 & 1 \\ 3 & 3 \\ 3 & 1 \end{pmatrix}. \qquad \boldsymbol{D}^{(-6)} = \begin{pmatrix} 1 & 3 \\ 3 & 1 \\ 3 & 1 \\ 3 & 2 \\ 1 & 5 \\ 1 & 1 \end{pmatrix}.$$

Both down-weights are $\frac{1}{2}$, so

$$\alpha_4 = \alpha_6 = \tan\left(\frac{\pi(1 - 1/2)}{2}\right) = 1.$$

Mixed prompt weighting — answer

Answer: For $s \in \{0, -4, -6\}$, set $\boldsymbol{C}^{(0)} = \boldsymbol{C}$ and calculate, for each $i = 1, \ldots, 6$,

$$\boldsymbol{d}_i^{(s)} = \boldsymbol{c}_i^{\varnothing} + r_i(\boldsymbol{c}_i^{(s)} - \boldsymbol{c}_i^{\varnothing}) \in \mathbb{R}^2.$$

For example, $\boldsymbol{d}_2^{(0)} = (1,1)^\top + 2((2,1)^\top - (1,1)^\top) = (3,1)^\top$.

$$\boldsymbol{D}^{(0)} = \begin{pmatrix} 1 & 3 \\ 3 & 1 \\ 3 & 1 \\ 3 & 2 \\ 1 & 5 \\ 2 & 3 \end{pmatrix}, \quad \boldsymbol{D}^{(-4)} = \begin{pmatrix} 1 & 3 \\ 3 & 1 \\ 3 & 1 \\ 1 & 1 \\ 3 & 3 \\ 3 & 1 \end{pmatrix}. \qquad \boldsymbol{D}^{(-6)} = \begin{pmatrix} 1 & 3 \\ 3 & 1 \\ 3 & 1 \\ 3 & 2 \\ 1 & 5 \\ 1 & 1 \end{pmatrix}.$$

Both down-weights are $\frac{1}{2}$, so

$$\alpha_4 = \alpha_6 = \tan\left(\frac{\pi(1 - 1/2)}{2}\right) = 1.$$

Thus every output row is the average of the corresponding three weighted rows:

Prompt weighting - Answer

Question: Can't we modify the information using vector operations?

Answer: Using the raw context vectors and those with empty prompts or masked attentions, we can strengthen or weaken the prompt's effect.

Outline

1. Introduction

2. Mathematical notation

3. Text encoder of Stable Diffusion 1.5

4. Can’t we use vector operations to modify the prompt information? - Prompt weighting.

5. Text encoder of Qwen-Image-2.1

6. Summary

Outline

1. Introduction

2. Mathematical notation

3. Text encoder of Stable Diffusion 1.5

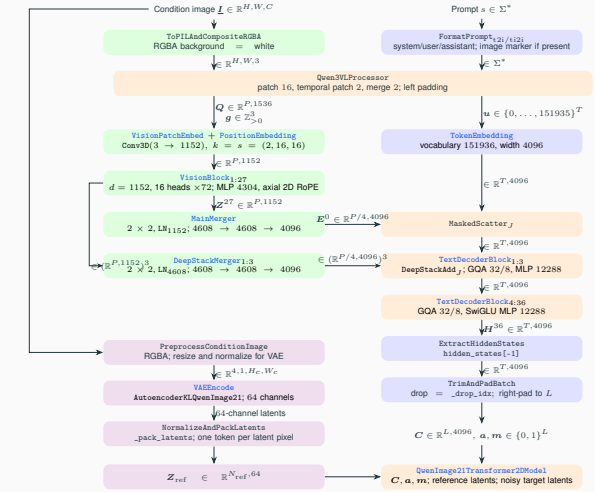
4. Can’t we use vector operations to modify the prompt information? - Prompt weighting.

5. Text encoder of Qwen-Image-2.1

6. Summary

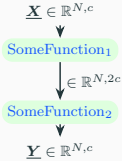
Question: How does the text encoder of Qwen-Image-2.1 convert users' prompt for the iterative generator? Why is there no prompt length constraint?

Qwen-Image-2.1 text encoder: Qwen3-VL-8B - Whole structure



Sources: text config; image config; processor; model; pipeline.

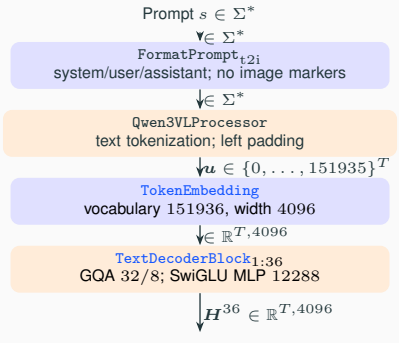
Legend



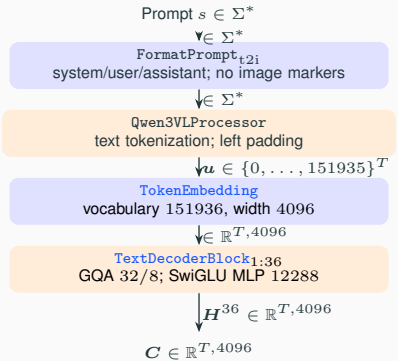
Colored boxes: functions. Arrows: tensors; annotations: tensor shapes.

RoPE are used as position embedding.

Qwen-Image-2.1 text encoder: Qwen3-VL-8B - Text-only case

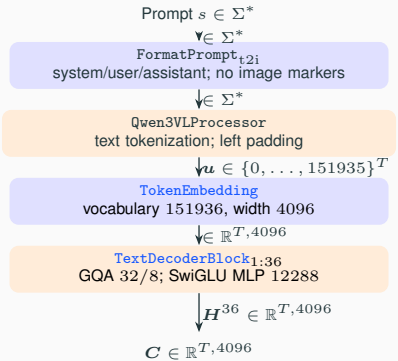


Qwen-Image-2.1 text encoder: Qwen3-VL-8B - Text-only case



- `TextDecoderBlock` is an attention based layer.

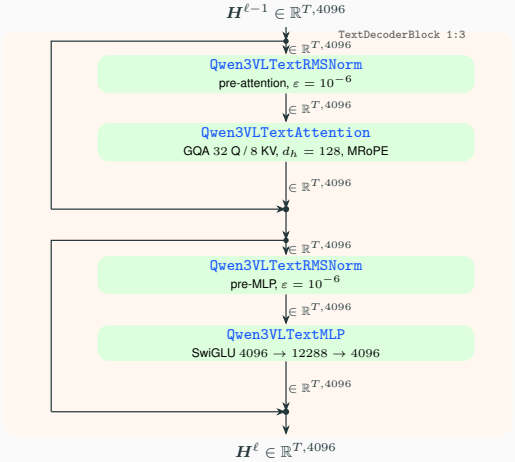
Qwen-Image-2.1 text encoder: Qwen3-VL-8B - Text-only case



Sources: text config; processor; model; pipeline.

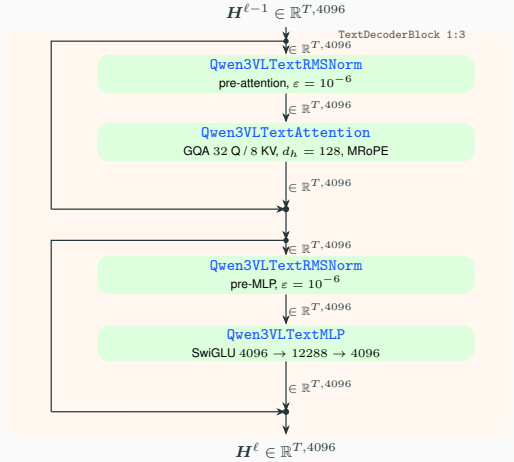
- `TextDecoderBlock` is an attention based layer.
- As position embedding vectors, it only uses **RoPE**, not learnable vectors.

TextDecoderBlock



The residual is given by:

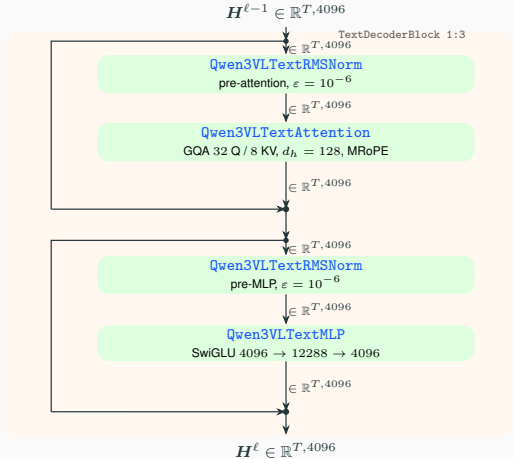
TextDecoderBlock



The residual is given by:

- Grouped-query attention with RoPE

TextDecoderBlock

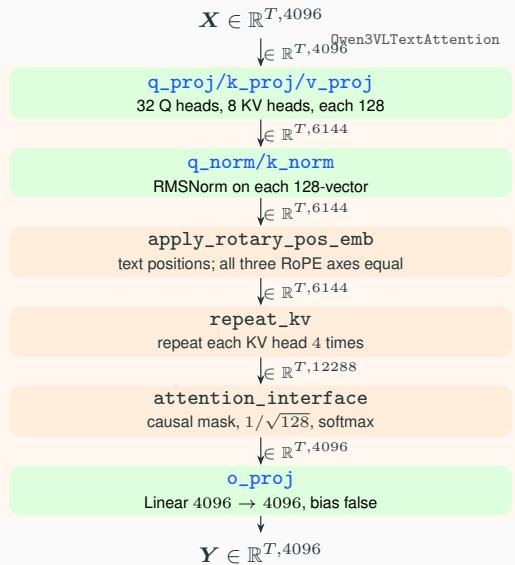


The residual is given by:

- Grouped-query attention with RoPE
- MLP

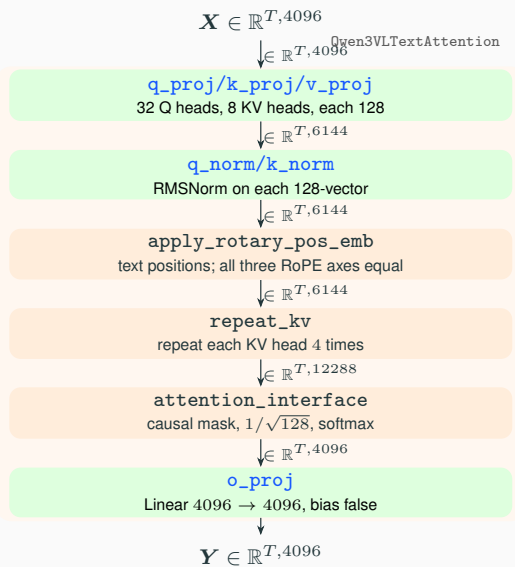
Source: modeling_qwen3_vl.py; text_encoder/config.json.

Qwen3VLTextAttention



It's an attention layer.

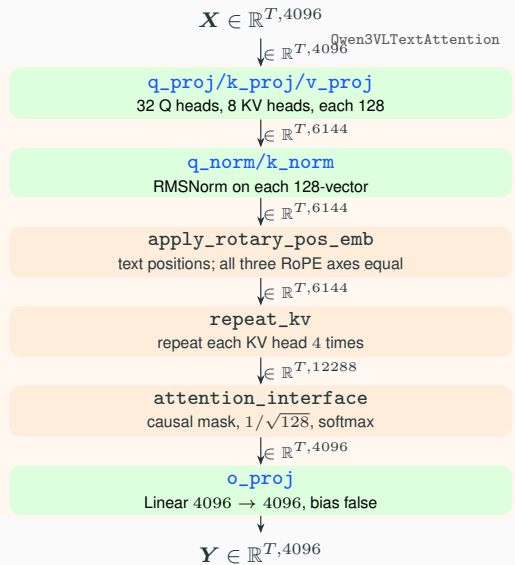
Qwen3VLTextAttention



It's an attention layer.

- Create 32 Q heads, 8 K heads, 8 V heads.

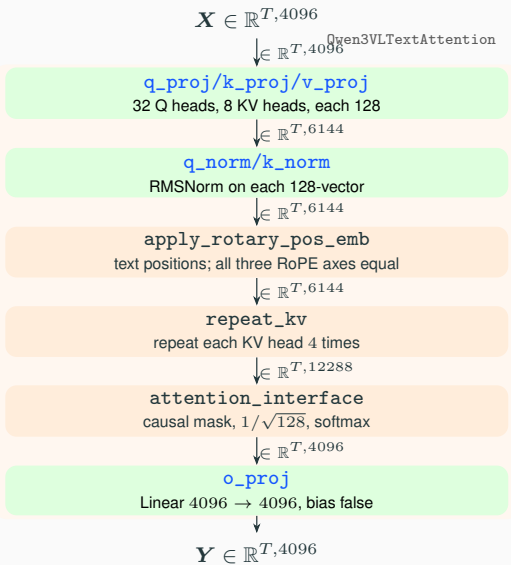
Qwen3VLTextAttention



It's an attention layer.

- Create 32 Q heads, 8 K heads, 8 V heads.
- Apply RoPE

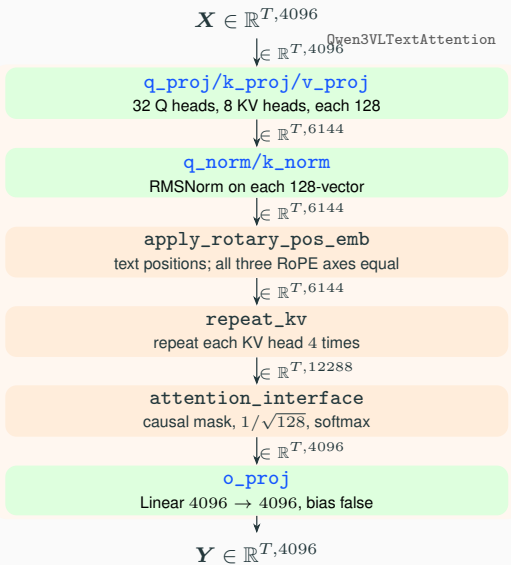
Qwen3VLTextAttention



It's an attention layer.

- Create 32 Q heads, 8 K heads, 8 V heads.
- Apply RoPE
- Apply the scaled-dot attention with K and V reused.

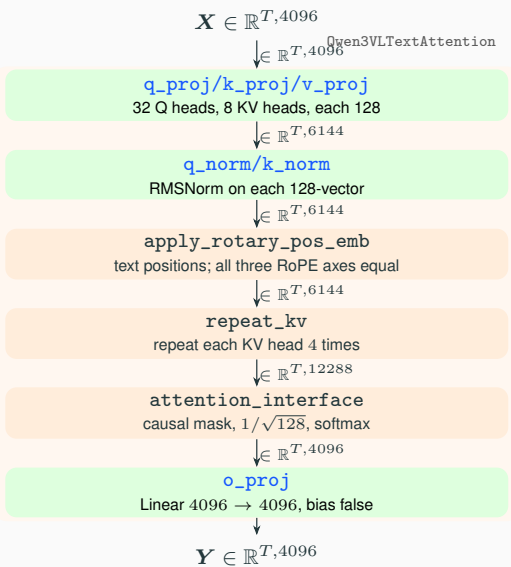
Qwen3VLTextAttention



It's an attention layer.

- Create 32 Q heads, 8 K heads, 8 V heads.
- Apply RoPE
- Apply the scaled-dot attention with K and V reused.
- Linear projection to create the output.

Qwen3VLTextAttention



It's an attention layer.

- Create 32 Q heads, 8 K heads, 8 V heads.
- Apply RoPE
- Apply the scaled-dot attention with K and V reused.
- Linear projection to create the output.

Notable points:

- Grouped-query attention
- Position embedding is RoPE.

Qwen3VLTextAttention: Grouped-query attention with RoPE

Input $(\boldsymbol{x}_0, \ldots, \boldsymbol{x}_{T-1})^\top \in \mathbb{R}^{T,4096}$. Params $\boldsymbol{W}_{\mathsf{Q}}^{[h]}, \boldsymbol{W}_{\mathsf{K}}^{[a]}, \boldsymbol{W}_{\mathsf{V}}^{[a]} \in \mathbb{R}^{128,4096}$, $\boldsymbol{W}_{\mathsf{O}} \in \mathbb{R}^{4096\times 4096}$, $\boldsymbol{\gamma}_{\mathsf{Q}}, \boldsymbol{\gamma}_{\mathsf{K}} \in \mathbb{R}^{128}$, for head index $h = 0, 1, \ldots, 31$, head group index $a = 0, 1, \ldots, 7$.

Projections and RMSNorm. For $i = 0, 1, \ldots, T-1$,

$$\boldsymbol{q}_i^{[h]} = \mathsf{RMSNorm}_{\boldsymbol{\gamma}_{\mathsf{Q}}}\left(\boldsymbol{W}_{\mathsf{Q}}^{[h]}\boldsymbol{x}_i\right) \in \mathbb{R}^{128},$$
$$\boldsymbol{k}_i^{[a]} = \mathsf{RMSNorm}_{\boldsymbol{\gamma}_{\mathsf{K}}}\left(\boldsymbol{W}_{\mathsf{K}}^{[a]}\boldsymbol{x}_i\right) \in \mathbb{R}^{128},$$
$$\boldsymbol{v}_i^{[a]} = \boldsymbol{W}_{\mathsf{V}}^{[a]}\boldsymbol{x}_i \in \mathbb{R}^{128}.$$

Rotary embeddings.

$$\tilde{\boldsymbol{q}}_i^{[h]} = \mathsf{RoPE}_{\boldsymbol{p}_i}(\boldsymbol{q}_i^{[h]}), \qquad \tilde{\boldsymbol{k}}_i^{[a]} = \mathsf{RoPE}_{\boldsymbol{p}_i}(\boldsymbol{k}_i^{[a]}),$$

where $\boldsymbol{p}_i = (i, i, i)^\top$.

Causal grouped-query attention. Let $\boldsymbol{Q}^{[h]}, \boldsymbol{K}^{[a]}, \boldsymbol{V}^{[a]}$ be the row-stacks of $\tilde{\boldsymbol{q}}_i^{[h]}, \tilde{\boldsymbol{k}}_i^{[a]}, \boldsymbol{v}_i^{[a]}$, respectively. For each head h,

$$\boldsymbol{Z}^{[h]} = \mathsf{ScaledDotAttention}\big(\boldsymbol{Q}^{[h]}, \boldsymbol{K}^{[a(h)]}, \boldsymbol{V}^{[a(h)]}, \boldsymbol{M}\big),$$

where $\boldsymbol{Z}^{[h]} = (\boldsymbol{z}_i^{[h]\top})_{i=0}^{T-1}$. where $\boldsymbol{M}$ is the causal mask and $a(h) = \lfloor h/4 \rfloor$ assigns four query heads to each key/value head.

Output projection.

$$\boldsymbol{y}_i = \boldsymbol{W}_{\mathsf{O}}(\boldsymbol{z}_i^{[0]\top}, \ldots, \boldsymbol{z}_i^{[31]\top})^\top \in \mathbb{R}^{4096}.$$

Source: `modeling_qwen3_vl.py`: `Qwen3VLTextAttention`, `eager_attention_forward`; text_encoder/config.json.

Qwen3VLTextAttention: RoPE, attention, and head groups

One-dimensional RoPE for text tokens. For token position $i \in \{0, \dots, T-1\}$, define $\omega_j = (5 \cdot 10^6)^{-2j/128}$ for $j = 0, \dots, 63$ and

$$R(\phi) = \begin{pmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{pmatrix}.$$

For $\boldsymbol{x} \in \mathbb{R}^{128}$, define $\mathrm{RoPE}_i(\boldsymbol{x}) = \tilde{\boldsymbol{x}}$ by

$$\begin{pmatrix} \tilde{x}_j \\ \tilde{x}_{j+64} \end{pmatrix} = R(i\omega_j) \begin{pmatrix} x_j \\ x_{j+64} \end{pmatrix}, \qquad j = 0, \dots, 63.$$

The 128 coordinates form 64 rotation pairs. For text tokens, $\boldsymbol{p}_i = (i, i, i)^\top$; hence $\mathrm{RoPE}_{\boldsymbol{p}_i} = \mathrm{RoPE}_i$.

Scaled dot-product attention. Let $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{T \times d}$, $d = 128$, and $\boldsymbol{M} = (m_{ij}) \in \{0, -\infty\}^{T \times T}$.

$$\begin{aligned} &\mathrm{ScaledDotAttention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}, \boldsymbol{M}) \\ &= \mathrm{softmax}_{\mathrm{row}}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d}} + \boldsymbol{M} \right) \boldsymbol{V}. \end{aligned}$$

For causal attention, $m_{ij} = 0$ when $j \le i$ and the key is valid, and $m_{ij} = -\infty$ otherwise.

Four queries per KV head.

$$a(h) = \left\lfloor \frac{h}{4} \right\rfloor, \quad h = 0, \dots, 31.$$

h	0	1	2	3	4	5	6	7	$\cdots$	31
$a(h)$	0	0	0	0	1	1	1	1	$\cdots$	7

Source: `Qwen3VLTextRotaryEmbedding`, `Qwen3VLTextAttention`; text configuration.

RoPE matrix computation — exercise

Exercise

Consider a toy one-dimensional RoPE on $\mathbb{R}^{12}$ *with six rotation pairs* (x_j, x_{j+6}), $j = 0, \ldots, 5$. *Use the prescribed frequencies* $\boldsymbol{\omega} = \frac{\pi}{6}(1, 2, 3, 2, 1, 2)^\top$. *At text position* $i \in \{0, 1, 2\}$, *set* $\phi_{i,j} = i\omega_j$ *and*

$$\boldsymbol{R}_i = \begin{pmatrix} \boldsymbol{D}_{c,i} & -\boldsymbol{D}_{s,i} \\ \boldsymbol{D}_{s,i} & \boldsymbol{D}_{c,i} \end{pmatrix}, \qquad \begin{aligned} \boldsymbol{D}_{c,i} &= \operatorname{diag}(\cos\phi_{i,0}, \ldots, \cos\phi_{i,5}), \\ \boldsymbol{D}_{s,i} &= \operatorname{diag}(\sin\phi_{i,0}, \ldots, \sin\phi_{i,5}). \end{aligned}$$

Let $\boldsymbol{x} = (1, 2, 0, -1, 1, 0,\ 0, 1, -1, 0, 2, 1)^\top$. *For each* $i = 0, 1, 2$, *calculate the angles* $\phi_{i,j}$, *the diagonal matrices* $\boldsymbol{D}_{c,i}, \boldsymbol{D}_{s,i}$, *and the rotated vector* $\boldsymbol{R}_i\boldsymbol{x}$.

Source for the paired-coordinate and interleaved-axis structure: `Qwen3VLTextRotaryEmbedding`, `apply_rotary_pos_emb`.

RoPE matrix computation — answer

Answer:

RoPE matrix computation — answer

Answer: Write $\boldsymbol{x} = (\boldsymbol{u}^\top, \boldsymbol{v}^\top)^\top$, $\boldsymbol{u} = (1, 2, 0, -1, 1, 0)^\top$, $\boldsymbol{v} = (0, 1, -1, 0, 2, 1)^\top$. For each i, set $\boldsymbol{\phi}_i = i\boldsymbol{\omega}$, $\boldsymbol{c}_i = (\cos \phi_{i,j})_{j=0}^5$, $\boldsymbol{s}_i = (\sin \phi_{i,j})_{j=0}^5$; then $\boldsymbol{D}_{c,i} = \mathrm{diag}(\boldsymbol{c}_i)$ and $\boldsymbol{D}_{s,i} = \mathrm{diag}(\boldsymbol{s}_i)$. Writing $\boldsymbol{R}_i \boldsymbol{x} = (\boldsymbol{u}_i^\top, \boldsymbol{v}_i^\top)^\top$ gives $\boldsymbol{u}_i = \boldsymbol{D}_{c,i}\boldsymbol{u} - \boldsymbol{D}_{s,i}\boldsymbol{v}$ and $\boldsymbol{v}_i = \boldsymbol{D}_{s,i}\boldsymbol{u} + \boldsymbol{D}_{c,i}\boldsymbol{v}$.

RoPE matrix computation — answer

Answer: Write $\boldsymbol{x} = (\boldsymbol{u}^\top, \boldsymbol{v}^\top)^\top$, $\boldsymbol{u} = (1, 2, 0, -1, 1, 0)^\top$, $\boldsymbol{v} = (0, 1, -1, 0, 2, 1)^\top$. For each i, set $\boldsymbol{\phi}_i = i\boldsymbol{\omega}$, $\boldsymbol{c}_i = (\cos \phi_{i,j})_{j=0}^5$, $\boldsymbol{s}_i = (\sin \phi_{i,j})_{j=0}^5$; then $\boldsymbol{D}_{c,i} = \operatorname{diag}(\boldsymbol{c}_i)$ and $\boldsymbol{D}_{s,i} = \operatorname{diag}(\boldsymbol{s}_i)$. Writing $\boldsymbol{R}_i \boldsymbol{x} = (\boldsymbol{u}_i^\top, \boldsymbol{v}_i^\top)^\top$ gives $\boldsymbol{u}_i = \boldsymbol{D}_{c,i}\boldsymbol{u} - \boldsymbol{D}_{s,i}\boldsymbol{v}$ and $\boldsymbol{v}_i = \boldsymbol{D}_{s,i}\boldsymbol{u} + \boldsymbol{D}_{c,i}\boldsymbol{v}$.

For $i = 0$: $\boldsymbol{\phi}_0 = \boldsymbol{0}$, $\boldsymbol{c}_0 = (1, 1, 1, 1, 1, 1)^\top$, $\boldsymbol{s}_0 = \boldsymbol{0}$ and $\boldsymbol{R}_0 \boldsymbol{x} = \boldsymbol{x}$.

RoPE matrix computation — answer

Answer: Write $\boldsymbol{x} = (\boldsymbol{u}^\top, \boldsymbol{v}^\top)^\top$, $\boldsymbol{u} = (1, 2, 0, -1, 1, 0)^\top$, $\boldsymbol{v} = (0, 1, -1, 0, 2, 1)^\top$. For each i, set $\boldsymbol{\phi}_i = i\boldsymbol{\omega}$, $\boldsymbol{c}_i = (\cos \phi_{i,j})_{j=0}^5$, $\boldsymbol{s}_i = (\sin \phi_{i,j})_{j=0}^5$; then $\boldsymbol{D}_{c,i} = \operatorname{diag}(\boldsymbol{c}_i)$ and $\boldsymbol{D}_{s,i} = \operatorname{diag}(\boldsymbol{s}_i)$. Writing $\boldsymbol{R}_i \boldsymbol{x} = (\boldsymbol{u}_i^\top, \boldsymbol{v}_i^\top)^\top$ gives $\boldsymbol{u}_i = \boldsymbol{D}_{c,i}\boldsymbol{u} - \boldsymbol{D}_{s,i}\boldsymbol{v}$ and $\boldsymbol{v}_i = \boldsymbol{D}_{s,i}\boldsymbol{u} + \boldsymbol{D}_{c,i}\boldsymbol{v}$.

For $i = 0$: $\boldsymbol{\phi}_0 = \boldsymbol{0}$, $\boldsymbol{c}_0 = (1, 1, 1, 1, 1, 1)^\top$, $\boldsymbol{s}_0 = \boldsymbol{0}$ and $\boldsymbol{R}_0 \boldsymbol{x} = \boldsymbol{x}$.

For $i = 1$: $\boldsymbol{\phi}_1 = \frac{\pi}{6}(1, 2, 3, 2, 1, 2)^\top$. Thus

$$\begin{pmatrix} \boldsymbol{c}_1^\top \\ \boldsymbol{s}_1^\top \end{pmatrix} = \begin{pmatrix} \sqrt{3}/2 & 1/2 & 0 & 1/2 & \sqrt{3}/2 & 1/2 \\ 1/2 & \sqrt{3}/2 & 1 & \sqrt{3}/2 & 1/2 & \sqrt{3}/2 \end{pmatrix},$$

RoPE matrix computation — answer

Answer: Write $\boldsymbol{x} = (\boldsymbol{u}^\top, \boldsymbol{v}^\top)^\top$, $\boldsymbol{u} = (1, 2, 0, -1, 1, 0)^\top$, $\boldsymbol{v} = (0, 1, -1, 0, 2, 1)^\top$. For each i, set $\boldsymbol{\phi}_i = i\boldsymbol{\omega}$, $\boldsymbol{c}_i = (\cos \phi_{i,j})_{j=0}^5$, $\boldsymbol{s}_i = (\sin \phi_{i,j})_{j=0}^5$; then $\boldsymbol{D}_{c,i} = \operatorname{diag}(\boldsymbol{c}_i)$ and $\boldsymbol{D}_{s,i} = \operatorname{diag}(\boldsymbol{s}_i)$. Writing $\boldsymbol{R}_i \boldsymbol{x} = (\boldsymbol{u}_i^\top, \boldsymbol{v}_i^\top)^\top$ gives $\boldsymbol{u}_i = \boldsymbol{D}_{c,i}\boldsymbol{u} - \boldsymbol{D}_{s,i}\boldsymbol{v}$ and $\boldsymbol{v}_i = \boldsymbol{D}_{s,i}\boldsymbol{u} + \boldsymbol{D}_{c,i}\boldsymbol{v}$.

For $i = 0$: $\boldsymbol{\phi}_0 = \boldsymbol{0}$, $\boldsymbol{c}_0 = (1, 1, 1, 1, 1, 1)^\top$, $\boldsymbol{s}_0 = \boldsymbol{0}$ and $\boldsymbol{R}_0 \boldsymbol{x} = \boldsymbol{x}$.

For $i = 1$: $\boldsymbol{\phi}_1 = \frac{\pi}{6}(1, 2, 3, 2, 1, 2)^\top$. Thus

$$\begin{pmatrix} \boldsymbol{c}_1^\top \\ \boldsymbol{s}_1^\top \end{pmatrix} = \begin{pmatrix} \sqrt{3}/2 & 1/2 & 0 & 1/2 & \sqrt{3}/2 & 1/2 \\ 1/2 & \sqrt{3}/2 & 1 & \sqrt{3}/2 & 1/2 & \sqrt{3}/2 \end{pmatrix},$$

$$\begin{pmatrix} \boldsymbol{u}_1^\top \\ \boldsymbol{v}_1^\top \end{pmatrix} = \begin{pmatrix} \sqrt{3}/2 & 1 - \sqrt{3}/2 & 1 & -1/2 & \sqrt{3}/2 - 1 & -\sqrt{3}/2 \\ 1/2 & \sqrt{3} + 1/2 & 0 & -\sqrt{3}/2 & \sqrt{3} + 1/2 & 1/2 \end{pmatrix}.$$

RoPE matrix computation — answer

Answer: Write $\boldsymbol{x} = (\boldsymbol{u}^\top, \boldsymbol{v}^\top)^\top$, $\boldsymbol{u} = (1, 2, 0, -1, 1, 0)^\top$, $\boldsymbol{v} = (0, 1, -1, 0, 2, 1)^\top$. For each i, set $\boldsymbol{\phi}_i = i\boldsymbol{\omega}$, $\boldsymbol{c}_i = (\cos \phi_{i,j})_{j=0}^5$, $\boldsymbol{s}_i = (\sin \phi_{i,j})_{j=0}^5$; then $\boldsymbol{D}_{c,i} = \operatorname{diag}(\boldsymbol{c}_i)$ and $\boldsymbol{D}_{s,i} = \operatorname{diag}(\boldsymbol{s}_i)$. Writing $\boldsymbol{R}_i \boldsymbol{x} = (\boldsymbol{u}_i^\top, \boldsymbol{v}_i^\top)^\top$ gives $\boldsymbol{u}_i = \boldsymbol{D}_{c,i}\boldsymbol{u} - \boldsymbol{D}_{s,i}\boldsymbol{v}$ and $\boldsymbol{v}_i = \boldsymbol{D}_{s,i}\boldsymbol{u} + \boldsymbol{D}_{c,i}\boldsymbol{v}$.

For $i = 0$: $\boldsymbol{\phi}_0 = \boldsymbol{0}$, $\boldsymbol{c}_0 = (1, 1, 1, 1, 1, 1)^\top$, $\boldsymbol{s}_0 = \boldsymbol{0}$ and $\boldsymbol{R}_0 \boldsymbol{x} = \boldsymbol{x}$.

For $i = 1$: $\boldsymbol{\phi}_1 = \frac{\pi}{6}(1, 2, 3, 2, 1, 2)^\top$. Thus

$$\begin{pmatrix} \boldsymbol{c}_1^\top \\ \boldsymbol{s}_1^\top \end{pmatrix} = \begin{pmatrix} \sqrt{3}/2 & 1/2 & 0 & 1/2 & \sqrt{3}/2 & 1/2 \\ 1/2 & \sqrt{3}/2 & 1 & \sqrt{3}/2 & 1/2 & \sqrt{3}/2 \end{pmatrix},$$
$$\begin{pmatrix} \boldsymbol{u}_1^\top \\ \boldsymbol{v}_1^\top \end{pmatrix} = \begin{pmatrix} \sqrt{3}/2 & 1 - \sqrt{3}/2 & 1 & -1/2 & \sqrt{3}/2 - 1 & -\sqrt{3}/2 \\ 1/2 & \sqrt{3} + 1/2 & 0 & -\sqrt{3}/2 & \sqrt{3} + 1/2 & 1/2 \end{pmatrix}.$$

For $i = 2$: $\boldsymbol{\phi}_2 = \frac{\pi}{6}(2, 4, 6, 4, 2, 4)^\top$. Thus

$$\begin{pmatrix} \boldsymbol{c}_2^\top \\ \boldsymbol{s}_2^\top \end{pmatrix} = \begin{pmatrix} 1/2 & -1/2 & -1 & -1/2 & 1/2 & -1/2 \\ \sqrt{3}/2 & \sqrt{3}/2 & 0 & \sqrt{3}/2 & \sqrt{3}/2 & \sqrt{3}/2 \end{pmatrix},$$

RoPE matrix computation — answer

Answer: Write $\boldsymbol{x} = (\boldsymbol{u}^\top, \boldsymbol{v}^\top)^\top$, $\boldsymbol{u} = (1, 2, 0, -1, 1, 0)^\top$, $\boldsymbol{v} = (0, 1, -1, 0, 2, 1)^\top$. For each i, set $\boldsymbol{\phi}_i = i\boldsymbol{\omega}$, $\boldsymbol{c}_i = (\cos \phi_{i,j})_{j=0}^5$, $\boldsymbol{s}_i = (\sin \phi_{i,j})_{j=0}^5$; then $\boldsymbol{D}_{c,i} = \operatorname{diag}(\boldsymbol{c}_i)$ and $\boldsymbol{D}_{s,i} = \operatorname{diag}(\boldsymbol{s}_i)$. Writing $\boldsymbol{R}_i \boldsymbol{x} = (\boldsymbol{u}_i^\top, \boldsymbol{v}_i^\top)^\top$ gives $\boldsymbol{u}_i = \boldsymbol{D}_{c,i}\boldsymbol{u} - \boldsymbol{D}_{s,i}\boldsymbol{v}$ and $\boldsymbol{v}_i = \boldsymbol{D}_{s,i}\boldsymbol{u} + \boldsymbol{D}_{c,i}\boldsymbol{v}$.

For $i = 0$: $\boldsymbol{\phi}_0 = \boldsymbol{0}$, $\boldsymbol{c}_0 = (1, 1, 1, 1, 1, 1)^\top$, $\boldsymbol{s}_0 = \boldsymbol{0}$ and $\boldsymbol{R}_0 \boldsymbol{x} = \boldsymbol{x}$.

For $i = 1$: $\boldsymbol{\phi}_1 = \frac{\pi}{6}(1, 2, 3, 2, 1, 2)^\top$. Thus

$$\begin{pmatrix} \boldsymbol{c}_1^\top \\ \boldsymbol{s}_1^\top \end{pmatrix} = \begin{pmatrix} \sqrt{3}/2 & 1/2 & 0 & 1/2 & \sqrt{3}/2 & 1/2 \\ 1/2 & \sqrt{3}/2 & 1 & \sqrt{3}/2 & 1/2 & \sqrt{3}/2 \end{pmatrix},$$
$$\begin{pmatrix} \boldsymbol{u}_1^\top \\ \boldsymbol{v}_1^\top \end{pmatrix} = \begin{pmatrix} \sqrt{3}/2 & 1 - \sqrt{3}/2 & 1 & -1/2 & \sqrt{3}/2 - 1 & -\sqrt{3}/2 \\ 1/2 & \sqrt{3} + 1/2 & 0 & -\sqrt{3}/2 & \sqrt{3} + 1/2 & 1/2 \end{pmatrix}.$$

For $i = 2$: $\boldsymbol{\phi}_2 = \frac{\pi}{6}(2, 4, 6, 4, 2, 4)^\top$. Thus

$$\begin{pmatrix} \boldsymbol{c}_2^\top \\ \boldsymbol{s}_2^\top \end{pmatrix} = \begin{pmatrix} 1/2 & -1/2 & -1 & -1/2 & 1/2 & -1/2 \\ \sqrt{3}/2 & \sqrt{3}/2 & 0 & \sqrt{3}/2 & \sqrt{3}/2 & \sqrt{3}/2 \end{pmatrix},$$
$$\begin{pmatrix} \boldsymbol{u}_2^\top \\ \boldsymbol{v}_2^\top \end{pmatrix} = \begin{pmatrix} 1/2 & -1 - \sqrt{3}/2 & 0 & 1/2 & 1/2 - \sqrt{3} & -\sqrt{3}/2 \\ \sqrt{3}/2 & \sqrt{3} - 1/2 & 1 & -\sqrt{3}/2 & 1 + \sqrt{3}/2 & -1/2 \end{pmatrix}.$$

Source for the rotation rule: `apply_rotary_pos_emb`.

RoPE for image - Interleaved 3-axis RoPE

For $\boldsymbol{p}_i = (p_{t,i}, p_{h,i}, p_{w,i})^\top$, let $\omega_j = (5 \cdot 10^6)^{-2j/128}$, $j = 0, \ldots, 63$, and

$$\rho(j) = \begin{cases} h, & j = 1, 4, \ldots, 58, \\ w, & j = 2, 5, \ldots, 59, \\ t, & \text{otherwise.} \end{cases}$$

Set $\phi_{i,j} = p_{\rho(j),i}\omega_j$ and $R(\phi) = \begin{pmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{pmatrix}$. For $\boldsymbol{x} \in \mathbb{R}^{128}$, define $\text{RoPE}_{\boldsymbol{p}_i}(\boldsymbol{x}) = \tilde{\boldsymbol{x}}$ by

$$\begin{pmatrix} \tilde{x}_j \\ \tilde{x}_{j+64} \end{pmatrix} = R(\phi_{i,j}) \begin{pmatrix} x_j \\ x_{j+64} \end{pmatrix}, \quad j = 0, \ldots, 63.$$

For text tokens, $\boldsymbol{p}_i = (i, i, i)^\top$.

Qwen3-VL: representative training data pairs

Input	Target text
Text prefix	Following text tokens
Image	Image caption or description
Image + question	Answer to the question
Video frames + question	Answer about the video
Interleaved text and images	Following text sequence

The text decoder is trained to predict target text tokens given the input context. These pairs describe task types, not the exact composition of Qwen-Image-2.1's text-encoder training set.

Source: Qwen3-VL Technical Report, Sections 3.1–3.2 (pre-training recipe and data).

Qwen-Image-2.1 text encoder - Answer

Question: How does the text encoder of Qwen-Image-2.1 convert users' prompt for the iterative generator? Why is there no prompt length constraint?

Answer:

Qwen-Image-2.1 text encoder - Answer

Question: How does the text encoder of Qwen-Image-2.1 convert users' prompt for the iterative generator? Why is there no prompt length constraint?

Answer:

- Qwen-Image 2.1's text encoder uses Transformer layers.

Qwen-Image-2.1 text encoder - Answer

Question: How does the text encoder of Qwen-Image-2.1 convert users' prompt for the iterative generator? Why is there no prompt length constraint?

Answer:

- Qwen-Image 2.1's text encoder uses Transformer layers.
- Qwen-Image 2.1 does not use learnable position embedding. It uses fixed RoPE. Hence, it does not suffer from the prompt length constraint.

Outline

1. Introduction

2. Mathematical notation

3. Text encoder of Stable Diffusion 1.5

4. Can’t we use vector operations to modify the prompt information? - Prompt weighting.

5. Text encoder of Qwen-Image-2.1

6. Summary

Outline

1. Introduction

2. Mathematical notation

3. Text encoder of Stable Diffusion 1.5

4. Can’t we use vector operations to modify the prompt information? - Prompt weighting.

5. Text encoder of Qwen-Image-2.1

6. Summary

Summary

- SD 1.5 uses the CLIP encoder. The architecture is attention-based. The position embedding is learnable, so it cannot accept prompts longer than 77 tokens.
- By prompt weighting, we can control the effect of the prompt information
- In Qwen-Image-2.1, the text encoder uses RoPE, so it does not suffer from the token length constraint.

References i

[1] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby.
An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale.
In International Conference on Learning Representations, 2021.

[2] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever.
Learning Transferable Visual Models From Natural Language Supervision.

References ii

In Proceedings of the 38th International Conference on Machine Learning, volume 139 of Proceedings of Machine Learning Research.